Is Code Better Than Language for Algorithmic Reasoning?

Terry Tong [1] **Yu Feng** [1] **Surbhi Goel** [1] **Dan Roth** [1]

Abstract

For tool-augmented language models, direct comparison between natural-language reasoning and code-execution pipelines are difficult to interpret because they conflate the intermediate representation and execution mechanisms. We resolve this ambiguity by inserting an intermediate counterfactual: the model expresses its reasoning as executable code, which is simulated in-context by the language model to produce an answer. Evaluations on 48 verifiable algorithmic benchmark tasks replicate the results in the literature where tool-calling language models outperform purely natural language reasoning models (+28.9pp). However, the latter performs similar to our intermediate counterfactual model, suggesting that, in our evaluated setting, interventions on the intermediate representation do not sufficiently explain the tool-use advantage. The results are consistent with the observation that gains in performance appear in conjunction with reliable external execution methods. A reconstruction intervention experiment using a proxy LM to infer the prompt + NL reasoning from the code representations performs similarly to the original NL reasoning on our algorithmic benchmark tasks. This provides qualitative (and statistical) evidence against the null hypothesis that representations cause better performance. We find that changes in execution mechanism explains most of the observed accuracy uplift. We formalize this intuition with a simple statistical decision-theoretic model that characterizes when execution dominates the end-to-end risk in our disentangled trace generation-execution regime. All experiments are at `https://github.com/TerryTong-Git/ToolProj`.

[*]Equal contribution [1]University of Pennsylvania. Correspondence to: Terry Tong <tongt1@seas.upenn.edu>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026.

1. Introduction

Many agentic systems orchestrate symbolic solvers, LLMs, and other tools, demonstrating state-of-the-art performance (Gao et al., 2023; Yao et al., 2023; Schick et al., 2023; Yang et al., 2024; Wang et al., 2025). Prior research empirically demonstrates that translating problems into solver-executable code representations (hereafter denoted Route 3) and delegating execution often outperforms reasoning end-to-end in NL (hereafter denoted Route 1) on logic- and algorithmic-style complex reasoning tasks (Lyu et al., 2023; Pan et al., 2023). However, it is unclear whether benefits come from the more structured code representation itself used in the planning and reasoning process, or the reliability of external solvers, or both. Little progress has been made in this direction since the problem itself is ill-posed: two routes learn fundamentally different objects, so there is no common formal target and metric to compare.

This paper presents a systematic three-route framework (Figure 1) breaks down the question into tractable pairwise comparisons between the different routes. We decouple the end-to-end pipeline into a trace generation phase (e.g. Chain of Thought (Wei et al., 2022)) and an execution phase (see Section 2.2). This allows us to fix one phase, and make causal claims on the effects of phase on end-to-end reasoning performance. We first instantiate this framework to verify the overall ordering Route 1 (NL + NL Reasoning) $\leq$ Route 2 (Code + NL Reasoning) $<$ Route 3 (Code + Python Execution) holds (Section 3). Solver-based execution results (+28.9% v.s. NL reasoning) in best performance across different models (`Mistral` (Jiang et al., 2023), `OpenAI` (Achiam et al., 2023), `Gemini` (Team et al., 2023), `Claude` (Anthropic, 2024)), tasks (`CLRS30` (Veličković et al., 2022), `NP-Hard-Eval` (Fan et al., 2023), Custom Eval Suite) and seeds $\{0, 1, 2\}$. Our results suggest Route 3 (Code + Python Execution) is optimal (Figures 3 and 4). To understand where language is bottlenecked, we operationalize our framework below.

When analyzing the pair Route 1 v.s. Route 2 (Section 4), we fix the execution phase to be a LLM with NL reasoning, and vary the representation modality of its Chain-of-Thought to be either code or NL. Because our goal is to optimize the end-to-end reasoning pipeline rather than any particular instantiation of it, we evaluate representations and executors in terms of their computation-constrained optimal risk

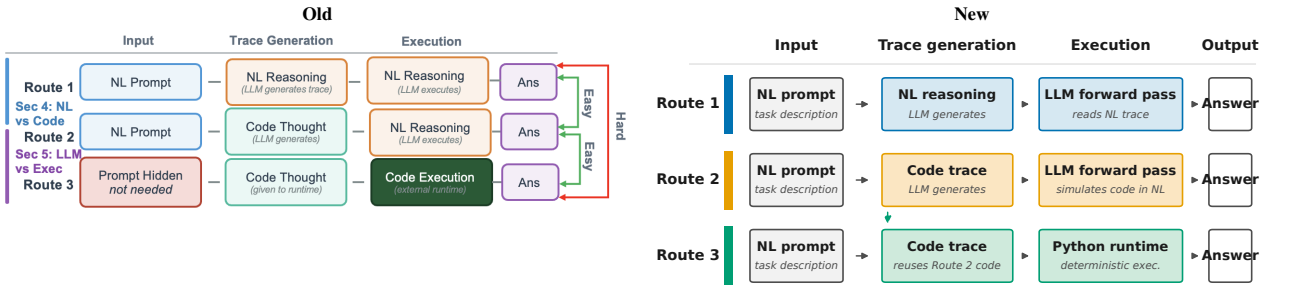

Figure 1. **Three-Route Framework.** We decompose algorithmic reasoning into: (1) *Translation* into NL or code, and (2) *Execution* via LLM or solver. This yields three routes: **Route 1** (Direct NL), **Route 2** (Code + NL simulation), **Route 3** (Code + Solver Execution). Prior work compares only Route 1 vs. Route 3, confounding translation and execution. Our Route 2 isolates these factors.

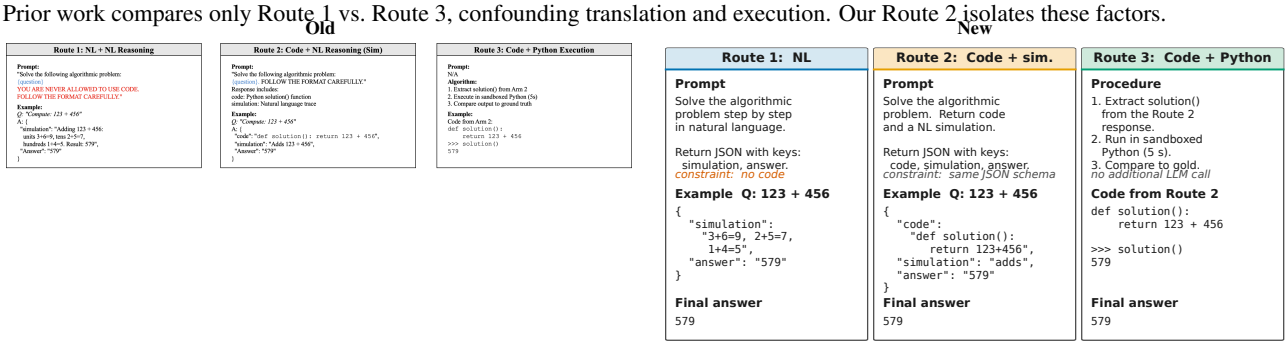

Figure 2. **Prompt templates for three-route evaluation. Route 1 (NL)**: LLM reasons in natural language only, code forbidden. **Route 2 (Sim)**: LLM generates Python `solution()` then simulates execution in NL. **Route 3 (Code)**: Same code executed in Python runtime. This isolates execution mechanism while controlling translation.

(hereafter optimal risk; Section 2.3). This criterion captures the best achievable performance within a model family and abstracts away suboptimal prompt or executor choices. We propose sufficient conditions such that the optimal risk of code is never worse than language up to a small amount of error ε (Proposition 4.2). Furthermore, we empirically verify these conditions through two experiments that show that natural language is an approximate garbling of code (Sections 4.1.1 and 4.1.2).

We further analyze Route 2 v.s. Route 3 (Section 5), identifying execution to be the bottleneck. We find that when code is correct, execution always achieves lower expected loss than language, and when code is incorrect, the probability that Route 2 $>$ Route 3 is small. On an analysis of hardness scaling, we find that code execution (Route 3) scales much better as problems become more difficult, and that the probabilities of Route 2 $>$ Route 3 when code is wrong reduces as problems get harder.

Our main contributions are:

1. A **three-route framework** (Section 2) for tractable comparison between code and natural language representations via an intermediary (code generation with LLM execution).
2. **Empirical validation** (Section 3) demonstrating that our framework's ordering holds such that Route 1 $\leq$ Route 2 $<$ Route 3.
3. A **systematic study** (Sections 4 and 5) ruling out trace generation, and solidifying execution as the bottleneck in end-to-end algorithmic reasoning performance when using language.

2. Three-Route Framework

Our central research question (RQ) is: *Is code $>$ NL for algorithmic reasoning?* To begin to answer this question, we first introduce our three-route framework which enables tractable comparison by disentangling *reasoning representation* (hereafter called Traces) and *reasoning execution* (hereafter called Executors) and constructing an intermediary bridge (Route 2) for pairwise comparison. This section introduces the task (Section 2.1), notation (Section 2.2), and definitions (Section 2.3, Section 2.4).

2.1. Task and Loss

For a gold evaluation distribution over task instances i specified by (algorithm, input variables, seed), let $X \sim p(x)$ denote a task instance (problem statement and inputs), and let $Y^*(X) \in \mathcal{Y}$ denote the ground-truth output that is unique, fixed, and externally verifiable. We evaluate performance under 0–1 loss:

$$\ell(y, x) := \mathbf{1}\{y \neq Y^*(x)\}.$$

All risks are taken with respect to the evaluation distribution $p(x)$.

2.2. Trace Generators and Executors

We model each reasoning pipeline as a two-stage stochastic process with *(1) trace generation* and *(2) execution*.

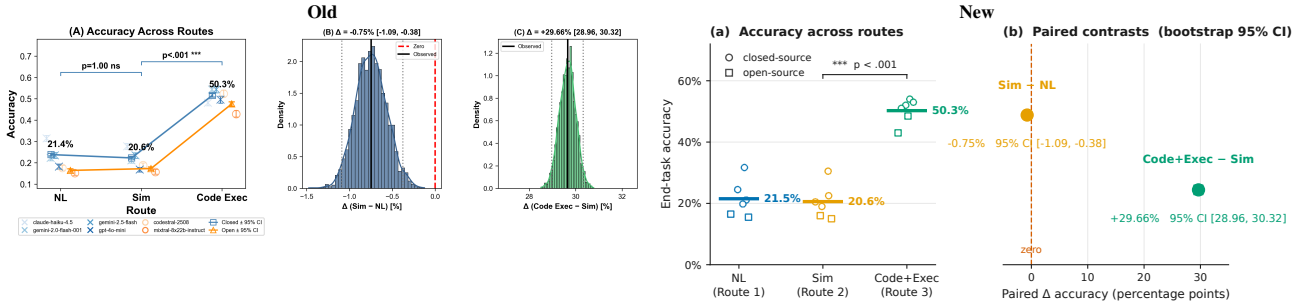

Figure 3. **Code + Solver Execution performs better than Direct NL reasoning quantified by end-task accuracy overall and within paired instances.** Paired instance contrasts are shown in the bootstrap distributions. Results are averaged over CLRS30, NPHardEval, Custom Eval Suite, 3 seeds and 6 models.

Trace generation. We define a *trace* as an object that stores intermediate reasoning used to solve a corresponding task (e.g. NL Chain-of-Thought or a program). A *trace generator* is a Markov kernel

$$E : \mathcal{X} \to \Delta(\mathcal{Z}), \qquad Z \sim p_E(z \mid x),$$

which produces an auxiliary trace Z given the task instance.

Execution. We define *execution* as a procedure that consumes a trace, e.g. an LLM forward pass that takes as input a reasoning trace and outputs an answer, or executing a program in an external runtime. An *executor* is a (possibly randomized) Markov kernel

$$\rho : \mathcal{X} \times \mathcal{Z} \to \Delta(\mathcal{Y}),$$

mapping the observed instance and trace to a final output. The induced conditional output distribution is

$$P_{\rho,E}(Y = y \mid X = x) = \int \rho(y \mid x, z)\, p_E(z \mid x)\, dz.$$

The population risk of a pipeline (E, ρ) is

$$R(E, \rho) := \mathbb{E}\big[\ell(\hat{Y}, X)\big],$$
$$Z \sim p_E(\cdot \mid X),\ \hat{Y} \sim \rho(\cdot \mid X, Z).$$

2.3. Computation-Constrained Optimal Risk

To reflect inference-time computational constraints, we evaluate executors within restricted families. For a trace space $\mathcal{Z}$ (e.g. code or NL), let

$$\mathcal{H}_{\mathcal{Z}} \subseteq \{\rho : \mathcal{X} \times \mathcal{Z} \to \Delta(\mathcal{Y})\}$$

denote a class of executors realizable under a fixed model family, e.g. Mistral (see Section 4.1). We define the computation-constrained optimal risk of a trace generator E as

$$R^*_{\mathcal{H}}(E) := \inf_{\rho \in \mathcal{H}} R(E, \rho).$$

This differs from classical Bayes risk, which optimizes over all measurable decision rules.

2.4. The Three Routes

We consider three reasoning pipelines ("routes"), illustrated in Figure 1. Each route is represented as a pair (E, ρ) consisting of a trace generator E and an executor ρ, evaluated via the risk $R(E, \rho)$ (Section 2.2) and the computation-constrained optimal risk $R^*_{\mathcal{H}}(E)$ (Section 2.3).

Route 1 (Direct Natural Language). Route 1 represents standard NL reasoning: the model first produces a natural-language (Chain-of-thought) trace and then the same model is used as the LLM-based executor, conditioning on the trace to generate a final answer. Formally, a natural-language trace generator E_{NL} produces traces $Z_{\mathrm{NL}} \sim p_{\mathrm{NL}}(\cdot \mid X)$, paired with an executor family

$$\mathcal{H}_{\mathrm{NL}} \subseteq \{\rho : \mathcal{X} \times \mathcal{Z}_{\mathrm{NL}} \to \Delta(\mathcal{Y})\}.$$

Route 2 (Code + NL Simulation). Route 2 uses *code* as the trace modality, but keeps execution "in-model": the LLM instead simulates the generated code in natural language, rather than executing it in an external environment. This route isolates the effect of using a code trace while holding the LLM-based executor class fixed. Formally, a code trace generator E_{Code} produces executable representations $Z_{\mathrm{C}} \sim p_{\mathrm{Code}}(\cdot \mid X)$, paired with an executor family

$$\mathcal{H}_{\mathrm{C}} \subseteq \{\rho : \mathcal{X} \times \mathcal{Z}_{\mathrm{C}} \to \Delta(\mathcal{Y})\}$$

corresponding to language-model-based simulation of code execution.

Route 3 (Code + Solver Execution). Route 3 uses the same code trace generator E_{Code} as Route 2, producing $Z_{\mathrm{C}} \sim p_{\mathrm{Code}}(\cdot \mid X)$, but pairs it with a deterministic executor corresponding to external code execution. Let $\mathrm{Exec} : \mathcal{X} \times \mathcal{Z}_{\mathrm{C}} \to \mathcal{Y}$ denote the (fixed) external runtime that executes the code trace z on instance x and returns an output. Our executor is

$$\rho_{\mathrm{Exec}}(y \mid x, z) := \mathbf{1}\{y = \mathrm{Exec}(x, z)\}.$$

The corresponding executor family is the singleton $\mathcal{H}_{\mathrm{Exec}} := \{\rho_{\mathrm{Exec}}\}$.

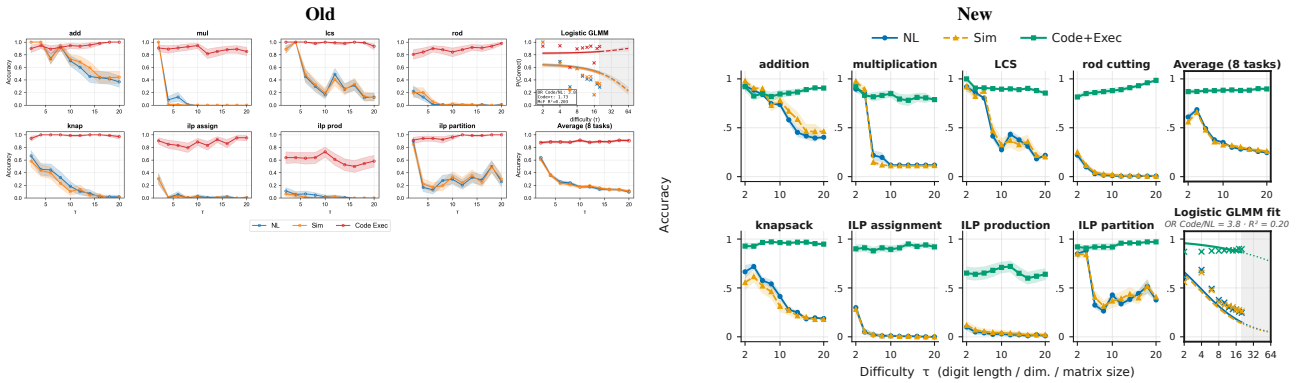

Figure 4. **Code + Solver Execution scales better than natural language reasoning when problems get harder, both within-task and on average, interpolating and extrapolating (grey) .** τ is used to control digit length for arithmetic, table dimensionality for dynamic programming, and constraint matrix dimensionality for integer linear programming. The same data and setup is used from Figure 3.

3. Evaluating the Three-Route Framework

Our evaluation aims to answer the research questions (RQ): 1) Does Route 1 $\leq$ Route 2 $<$ Route 3 hold? (§3.1, §3.2)

3.1. Experimental Instantiation of the Three Routes

To draw conclusions about the effect of modality in the trace generation or different execution methods, we ensure one stage is always fixed. For Route 1 vs Route 2, we fix the execution phase by using the same LLM reasoning model forward pass, but use different modalities in the trace generator. For Route 2 vs Route 3, we fix the trace generator which outputs code, then use different execution methods, either LLM reasoning model forward pass, or a python3 runtime. Below, let X_i be the task instantiation of instance i. Structured JSON outputs are enforced for sampling in all routes (Figure 2).

Sampling Route 1. Route 1 prompts an LLM E_{NL} to function as a trace generator $Z_{NL} := E_{NL}(X_i)$. The trace is then fed into the same LLM $\rho_{NL} = E_{NL}$ to produce an answer $Y_i^{(NL)} := \rho_{NL}(Z_{NL})$. The prompt instructs the model to never use code, and to output a structured rationale and answer (see Figure 2). Operationalized, one experimental run for Route 1 is encapsulated in a single LLM reasoning model's forward pass without pause.

Sampling Route 2. Similarly, Route 2 uses a LLM E_{Code} as a trace generator for code modality $Z_{Code} := E_{Code}(X_i)$. The trace is then fed into the same LLM $\rho_{Sim} = E_{Code}$ to produce an answer $Y_i^{(Sim)} := \rho_{Sim}(Z_{Code})$. Operationalized, one experimental run for Route 2 is encapsulated in a single LLM reasoning model's forward pass without pause. Prompt templates are controlled to be as similar as possible, with Route 2 differing only by the inclusion of a code-generation and simulation instruction (Figure 2). We prompt the model to output a program snippet, structured reasoning over the code, followed by an answer.

Sampling Route 3. Route 3 takes the exact same code trace Z_{Code} as Route 2, except it runs it through a python3 runtime ρ_{Exec}, and retrieves the solution $Y_i^{(Exec)} = \rho_{Exec}(Z_{Code})$. The python3 runtime has access to 5 standard scientific libraries: `Scipy`, `Numpy`, `Pandas`, `PuLP`, and `Pytorch`.

Observed outcomes. For each problem instance i, we observe paired Bernoulli outcomes

$$\left(Y_i^{(\mathrm{NL})},\ Y_i^{(\mathrm{Sim})},\ Y_i^{(\mathrm{Exec})}\right).$$

Data and models. We evaluate on the CLRS-30 benchmark ($n = 500$), the NP-Hard-Eval benchmark ($n = 540$), and a custom fine-grained evaluation suite ($n = 540$), across three random seeds $\{0, 1, 2\}$. Tasks span Arithmetic, Dynamic Programming, and Integer Linear Programming (ILP), with difficulty controlled by a parameter τ. We evaluate both closed-source models (`Claude Haiku 4.5`, `GPT-4o-mini`, `Gemini 2.5 Pro`) and open-source models (`Mixtral-8x22b-Instruct`, `Codestral-22B`).

3.2. End-to-End Performance Comparison

Pairwise statistical tests. We evaluate pairwise route differences using McNemar tests on paired Bernoulli outcomes. For Route 1 vs. Route 2, the null hypothesis is

$$H_0 :\ \Pr(Y^{(\mathrm{NL})} = 1,\ Y^{(\mathrm{Sim})} = 0) = \Pr(Y^{(\mathrm{NL})} = 0,\ Y^{(\mathrm{Sim})} = 1),$$

with an analogous null for Route 2 vs. Route 3.
Effect sizes are reported as paired accuracy differences, e.g.,

$$\Delta_{\mathrm{Sim-NL}} = \mathrm{Acc}(\mathrm{Sim}) - \mathrm{Acc}(\mathrm{NL}),$$

with 95% confidence intervals estimated via cluster bootstrap resampling over instances. Holm–Bonferroni correction is applied to control family-wise error rate of the above 2 marginal pairwise tests on fully pooled data at $\alpha = 5\%$.

Difficulty scaling. To analyze how performance varies with task difficulty, we fit a generalized linear mixed-effects model (GLMM) for binary accuracy $Y_i \in \{0, 1\}$ with a

logistic link:

$$Y_i \mid u_{\text{inst}[i]}, u_{\text{seed}[i]} \sim \text{Bernoulli}(p_i),$$

$$\text{logit}(p_i) = \alpha + \beta_{\text{route}_i} + \gamma\,\tau_i + \delta_{\text{route}_i}\,\tau_i + u_{\text{inst}[i]} + u_{\text{seed}[i]},$$

where $u_{\text{inst}[i]} \sim \mathcal{N}(0, \sigma^2_{\text{inst}})$ and $u_{\text{seed}[i]} \sim \mathcal{N}(0, \sigma^2_{\text{seed}})$ are random intercepts. Route and difficulty are modeled as fixed effects (including a route $\times$ difficulty interaction), with instance and seed modeled as random effects.

Results. Across all benchmarks (Figure 3), we reject the global null hypothesis ($p < 0.001$ after correction). *Post-hoc tests show a statistically significant advantage of Route 3 over Route 2*, with a positive paired accuracy gap excluding zero in the 95% confidence interval. For Route 1 vs. Route 2 we do not reject the null, indicating that we cannot conclude a meaningful difference between natural-language reasoning and code simulation under this evaluation. Performance gaps widen with increasing task difficulty (Figure 4): Route 3 remains stable while Route 1 degrades sharply, with code execution achieving approximately $4.1\times$ higher odds of correctness.

4. Route 1 vs Route 2 Analysis

In Section 3.2, Route 1 and Route 2 achieved similar paired accuracies under our evaluated models and prompts. The key research question in this section is, thus, to find the sufficient conditions: **Under what conditions are code traces at most ε worse in expected loss than NL traces over task distribution $P(X, Y^*)$, for a specified class of downstream executors?** We identify a sufficient criteria under which

$$R^*_{\mathcal{H}_\text{C}}(E_\text{Code}) \;\le\; R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \varepsilon.$$

holds.

Functional similarity condition. Let $E_\text{Tran} := T \circ E_\text{Code}$ be the NL-style trace generator obtained by translating code traces into NL traces. Assume that translated traces preserve downstream risk for the NL executor class:

$$\sup_{\rho \in \mathcal{H}_\text{NL}} |R(E_\text{NL}, \rho) - R(E_\text{Tran}, \rho)| \le \varepsilon.$$

Assume also that the code-side executor class can realize translation followed by any NL executor: for every $\rho \in \mathcal{H}_\text{NL}$, the composed executor $\rho \circ T$ belongs to $\mathcal{H}_\text{C}$.

Proposition 4.1. *Under the above conditions,*

$$R^*_{\mathcal{H}_\text{C}}(E_\text{Code}) \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \varepsilon.$$

Proof. Fix $\delta > 0$ and choose $\rho_\delta \in \mathcal{H}_\text{NL}$ such that

$$R(E_\text{NL}, \rho_\delta) \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \delta.$$

By realizability, the composed executor $\rho_\delta \circ T$ belongs to $\mathcal{H}_\text{C}$. Therefore,

$$R^*_{\mathcal{H}_\text{C}}(E_\text{Code}) \le R(E_\text{Code}, \rho_\delta \circ T).$$

By construction, running $\rho_\delta \circ T$ on code traces has the same risk as running ρ_δ on translated traces:

$$R(E_\text{Code}, \rho_\delta \circ T) = R(E_\text{Tran}, \rho_\delta).$$

By functional similarity,

$$R(E_\text{Tran}, \rho_\delta) \le R(E_\text{NL}, \rho_\delta) + \varepsilon \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \delta + \varepsilon.$$

Letting $\delta \to 0$ gives

$$R^*_{\mathcal{H}_\text{C}}(E_\text{Code}) \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \varepsilon.$$

□

Condition 1. (ε-garbling) We say that NL traces are ε-simulable from code traces (for the executor family $\mathcal{H}_\text{NL}$) if there exists a translation kernel $T : \mathcal{Z}_\text{C} \to \Delta(\mathcal{Z}_\text{NL})$ such that, letting $E_\text{Tran} := T \circ E_\text{Code}$,

$$\Delta_{\mathcal{H}_\text{NL}}(E_\text{NL}, E_\text{Tran}) := \sup_{\rho \in \mathcal{H}_\text{NL}} |R(E_\text{NL}, \rho) - R(E_\text{Tran}, \rho)| \le \varepsilon,$$

and such that $\mathcal{H}_\text{C}$ is closed under composition with T (i.e., for each $\rho_\text{NL} \in \mathcal{H}_\text{NL}$, $\rho_{T\circ\text{NL}} \in \mathcal{H}_\text{C}$). In §4.1.1 and §4.1.2 we estimate these quantities and find ε is small.

Proposition 4.2 (Consequence of ε-garbling)**.** *If Condition 1 holds, then*

$$R^*_{\mathcal{H}_\text{C}}(E_\text{Code}) \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \varepsilon.$$

Proof. Fix any $\delta > 0$ and choose $\rho^\delta_\text{NL} \in \mathcal{H}_\text{NL}$ such that

$$R(E_\text{NL}, \rho^\delta_\text{NL}) \le R^*_{\mathcal{H}_\text{NL}}(E_\text{NL}) + \delta.$$

By Condition 1, the composed executor that "translates then decodes,"

$$\rho^\delta_{T\circ\text{NL}}(y \mid x, z_\text{C}) := \int \rho^\delta_\text{NL}(y \mid x, z_\text{NL})\, T(z_\text{NL} \mid z_\text{C})\, dz_\text{NL},$$

is realizable in the code-executor class, i.e., $\rho^\delta_{T\circ\text{NL}} \in \mathcal{H}_\text{C}$. Now consider the following stochastic computation graph: first sample a code trace $Z_\text{C} \sim p_\text{Code}(\cdot \mid X)$; then translate it to an NL trace $Z_\text{NL} \sim T(\cdot \mid Z_\text{C})$; finally sample the answer $\hat{Y} \sim \rho^\delta_\text{NL}(\cdot \mid X, Z_\text{NL})$. This is exactly what $\rho^\delta_{T\circ\text{NL}}$ implements: it performs the translation step inside the executor and then applies the NL executor.
Equivalently, we may "push" the translation step into the trace generator. By construction of the translated generator $E_\text{Tran} := T \circ E_\text{Code}$, the marginal conditional distribution of Z_NL given X is

$$p_\text{Tran}(z_\text{NL} \mid x) = \int T(z_\text{NL} \mid z_\text{C})\, p_\text{Code}(z_\text{C} \mid x)\, dz_\text{C}.$$

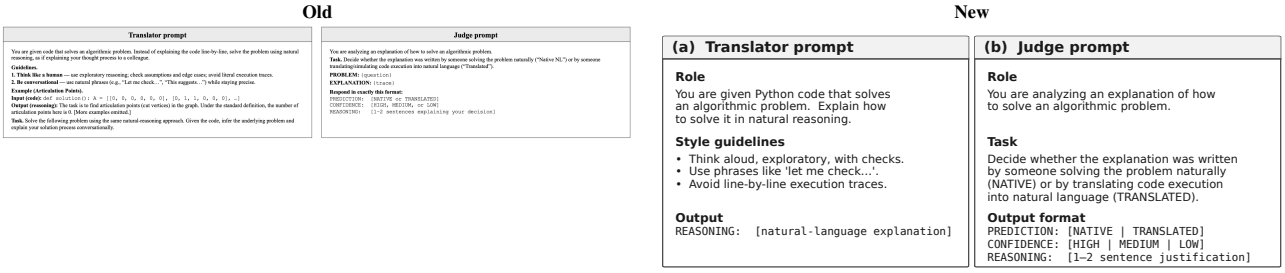

Figure 5. **Translation and discrimination prompts. (a) Translator**: Converts code to NL reasoning step-by-step, mimicking native reasoning. **(b) Discriminator**: Judge models classify traces as "Native NL" or "Translated". Used in Section 4.

Therefore, the joint distribution of $(X, Z_{\mathrm{NL}}, \hat{Y})$ induced by (i) $(E_{\mathrm{Code}}, \rho^{\delta}_{T\circ\mathrm{NL}})$ and (ii) $(E_{\mathrm{Tran}}, \rho^{\delta}_{\mathrm{NL}})$ is the same, and hence they have identical risk:

$$R(E_{\mathrm{Code}}, \rho^{\delta}_{T\circ\mathrm{NL}}) = R(E_{\mathrm{Tran}}, \rho^{\delta}_{\mathrm{NL}}).$$

By Condition 1,

$$R(E_{\mathrm{Tran}}, \rho^{\delta}_{\mathrm{NL}}) \le R(E_{\mathrm{NL}}, \rho^{\delta}_{\mathrm{NL}}) + \varepsilon \le R^{*}_{\mathcal{H}_{\mathrm{NL}}}(E_{\mathrm{NL}}) + \delta + \varepsilon.$$

Taking the infimum over $\mathcal{H}_{\mathrm{C}}$ and letting $\delta \to 0$ completes the proof. □

4.1. Validating Condition 1.

Condition 1 requires natural language to be an approximate garbling of code. To verify this, we ask the sub-questions as a proxy:

1) Does there exist a translator LLM such that the NL traces translated from code *look* like the original NL traces (existence of ε-Translator)? (§4.1.1)
2) Does there exist a translator LLM such that the NL traces translated from code *function* like the original NL traces quantified by end-task accuracy (closure under composition)? (§4.1.2)

In essence, these questions ask whether translation during inference really can be done in either trace generation or execution interchangeably (equal after marginalizing over code traces), similar to the ideas presented in the proof of Proposition 4.2. We find that a translator LLM does exist (e.g. `GPT-4o` as an example), such that the NL traces translated from code look and function like the original NL traces. This provides support for Condition 1 and suggests that NL does not provide decision-relevant information beyond code in the optimal risk regime.

4.1.1. Existence of ε-Translator

We find that the distribution of reasoning traces produced in Route 1 (cf. Section 2.4) can be approximated by post-processing code traces with a fixed transformation (`GPT-4o` with prompting). Concretely, for each task instance x and corresponding code c, we form two trace conditions:

$$(x, z_{\mathrm{NL}}), \quad z_{\mathrm{NL}} \sim p_{\mathrm{NL}}(\cdot \mid x), \quad (\textit{Native}),$$
$$(x, \hat{z}_{\mathrm{NL}}), \quad \hat{z}_{\mathrm{NL}} \sim p_{\mathrm{Tr}}(\cdot \mid c), \quad (\textit{Translated}),$$

where $p_{\mathrm{Tr}}(\cdot \mid c)$ is induced by a fixed translator T applied to code, i.e., $\hat{z}_{\mathrm{NL}} = T(c)$ with fixed prompting/in-context examples (Figure 5a).

Discrimination task. We formulate a binary classification task: a powerful zero-shot judge model is given (x, z) and must predict whether z is *Native* or *Translated* (Figure 5b). This corresponds to distinguishing the joint distributions

$$p(x)p_{\mathrm{NL}}(z \mid x) \quad \text{vs.} \quad p(x)p_{\mathrm{Tr}}(z \mid x).$$

We use multiple judge models (e.g., Claude Opus 4.0, Gemini 2.5 Pro, Grok 4.1 Fast) to reduce dependence on a single discriminator.

Sampling and controls. We sample problem instances x from a pool of previously collected CoT examples spanning different models[1] , seeds, tasks, and difficulties, and construct paired examples with balanced labels. Tasks used for discrimination are held out from the translator's in-context examples to avoid trivial leakage. As a calibration control, we also ask the same judges to discriminate *raw code* vs. *native NL* traces; high control accuracy indicates the judges are not underpowered.

Results. Across ∼2000 samples per judge (∼6000 total across three judges), discrimination accuracy is near chance (50%), with 50% included in the Wilson CI (Figures 6 and 7). The control discrimination between raw code and native NL is substantially higher (e.g., ≈82%), indicating the judges can detect obvious representation differences but *fail to distinguish translated vs. native NL.*

4.1.2. Closure under Translation

We wish to show that whatever NL-executor can do after seeing an NL trace, the NL-executor can do after seeing a code trace, by first translating the code trace into an NL trace then behaving like the original NL-executor. We therefore test functional similarity using a downstream accuracy intervention.

[1] In our experiments, when we conditioned on the models for the source inputs, we find that they become more distinguishable. This suggests that between models, there are stylistic artifacts that are significant enough to differentiate them. Since this is orthogonal to the native vs translated signal, we did not condition on models.

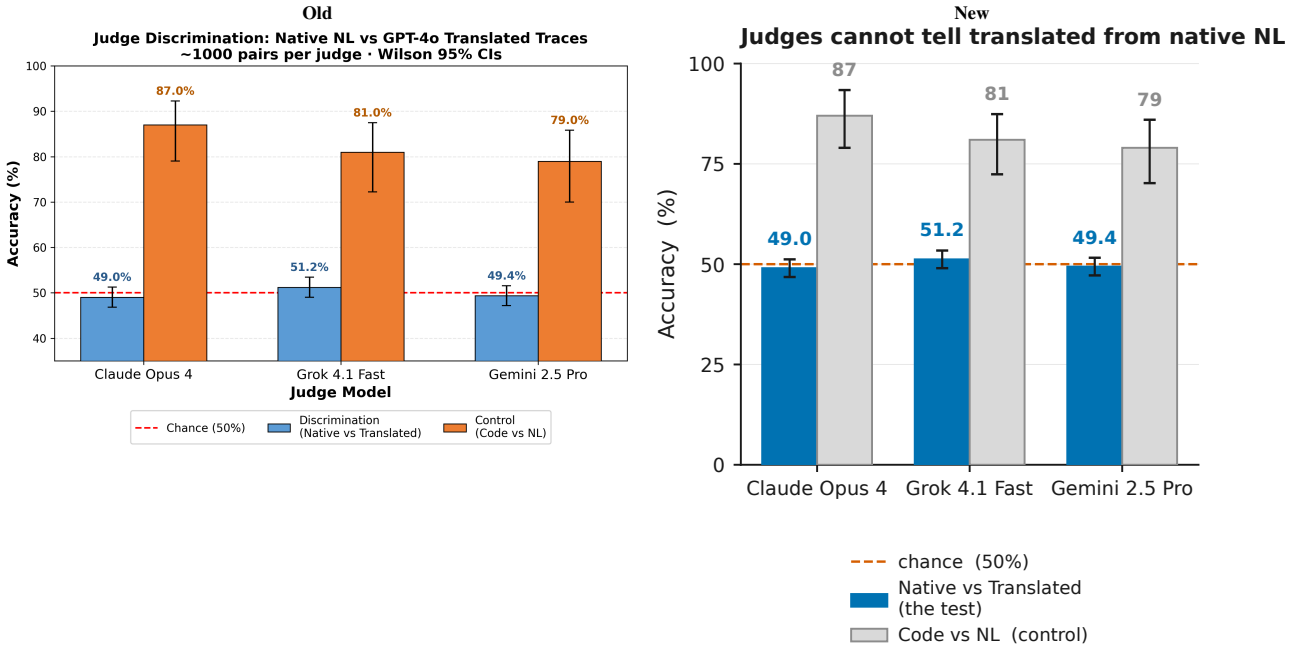

Figure 6. **Translated NL from code is indistinguishable to original NL traces, supporting the hypothesis that NL contains no more decision-related information than code.** Judges run zero-shot and are prompted to discriminate between native and translated, the translator is GPT-4o which gets 10-shot examples. Original data sources are pooled data from Figure 2.

Intervention setup. For a task instance x, we prompt a target language model in three conditions: (1) Baseline: x, (2) Native: $x \,\|\, z_{\mathrm{NL}}$, (3) Translated: x $\|\, \hat{z}_{\mathrm{NL}}$, where $z_{\mathrm{NL}} \sim p_{\mathrm{NL}}(\cdot \mid x)$ is the native Route 1 trace and $\hat{z}_{\mathrm{NL}}$ is obtained by translating the corresponding code to NL using the same translator procedure as in §4.1.1. If translated NL loses decision-relevant information relative to native NL, we would expect

$$\mathrm{Acc}(x \,\|\, \hat{z}_{\mathrm{NL}}) < \mathrm{Acc}(x \,\|\, z_{\mathrm{NL}}).$$

Protocol and models. We run this test on 1000 held-out instances across multiple translator models (`Claude-Haiku-4.5`, `Gemini-2.5-Flash`, `Mixtral-8x22B-Instruct`). Crucially, in this experiment the translator model is matched to the original code generator (i.e., we translate code produced by the same model family), to avoid confounding functional losses with cross-model stylistic mismatch.

Results. We fail to reject the null hypothesis of equal performance, and observe overlapping 95% confidence intervals between the Native and Translated conditions (Figure 8). This suggests that, under our evaluated settings, translating code into NL does not destroy decision-relevant information for downstream answering, consistent with the claim that NL CoT does not systematically add algorithmic advantage beyond what is in code. Qualitatively, we notice that the translated results are quite similar to the originals, though this seems heavily reliant on the translating prompt.

Interpretation. Our empirical results indicate that Condition 1 holds in the settings we study, implying that the best achievable performance using code traces matches that of natural-language traces up to a small error ε. In particular, we find no evidence that natural-language reasoning introduces novel algorithmic strategies beyond those already captured by code representations on algorithmic tasks. As a result, replacing NL traces with code traces in the generation stage does not constitute a performance bottleneck when optimizing the end-to-end reasoning pipeline. Instead, the remaining limitations are best attributed to the execution mechanism rather than the trace representation (Section 5).

5. Route 2 vs. Route 3 Analysis

The key research question in this section is: **Is execution the bottleneck for language-based reasoning?** To address this question, we also ask two sub-questions:

1) When code is *correct*, does external code execution (Route 3) have lower expected loss than LLM-based code execution (Route 2)?
2) When code is *incorrect*, is the probability that Route 2 outperforms Route 3 small?

We pinpoint execution as the bottleneck (cf. Section 4), highlighting how using LLMs to generate code plans and then delegating to an external solver is the optimal policy among the 3 routes (Section 3.2). We find that when code is correct, external code execution always has lower expected loss than LLM-based code execution. Moreover, when code is incorrect, we find the recovery rate, where Route 2 $>$ Route 3, is small.

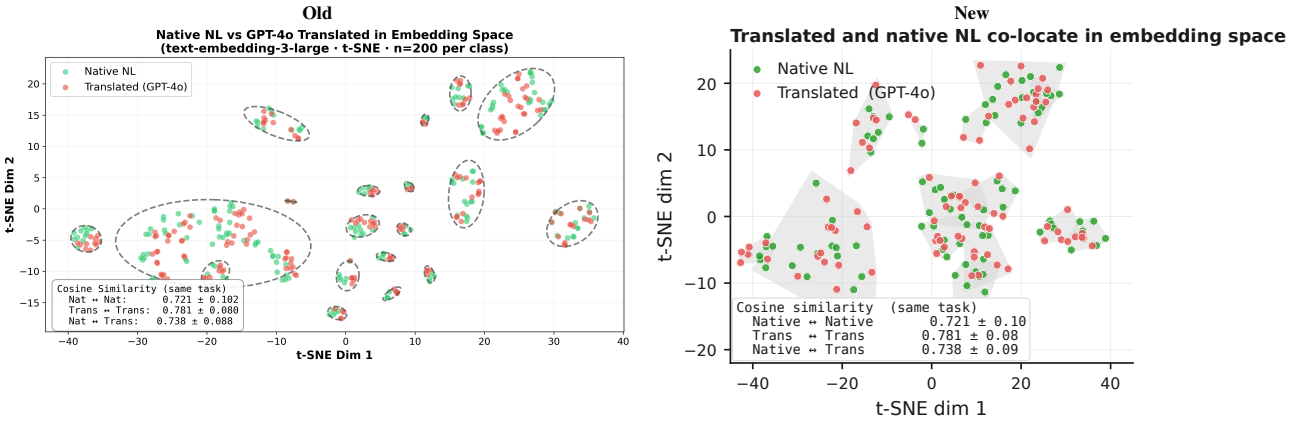

Figure 7. **Translated NL and Native NL form local clusters in the embedding space and have high cosine similarity, explaining distributional indistinguishability.** We randomly sample and embed 200 native NL reasoning traces and 200 code-to-NL translations (produced by GPT-4o) using OpenAI's text-embedding-3-large and project them via t-SNE. Cosine similarities between Native-Native and Translated-Translated sample pairs from the same task.

5.1. Setup

As defined in Section 2, Route 2 and Route 3 share the same trace generator E_{Code} and differ only in the executor. Let $\rho_{\text{Sim}} \in \mathcal{H}_{\text{C}}$ denote the fixed LLM-based simulation executor used in Route 2. Let g be a deterministic interpreter mapping (x, z_{C}) to $\mathcal{Y} \cup \{\perp\}$, where $\perp$ denotes execution failure. Define the instance-correct execution event

$$C := \{g(X, Z_{\text{C}}) = Y^*(X)\}.$$

Risk decomposition. Let

$$e_C := \Pr(\hat{Y}_{\text{Sim}} \neq Y^*(X) \mid C),$$
$$r := \Pr(\hat{Y}_{\text{Sim}} = Y^*(X) \mid \neg C).$$

Then

$$R(E_{\text{Code}}, \rho_{\text{Exec}}) = \Pr(\neg C),$$
$$R(E_{\text{Code}}, \rho_{\text{Sim}}) = \Pr(C)\, e_C + \Pr(\neg C)(1 - r),$$

and hence

$$R(E_{\text{Code}}, \rho_{\text{Sim}}) - R(E_{\text{Code}}, \rho_{\text{Exec}}) = \Pr(C)\, e_C - \Pr(\neg C)\, r.$$

Implications. Simulation noise on correct-execution instances ($\Pr(C)e_C$) harms Route 2, while recovery on incorrect or failed executions ($\Pr(\neg C)r$) helps Route 2. Route 3 dominates whenever the recovery mass is insufficient to offset simulation noise, a condition quantified empirically in Section 3.2.

Empirical recovery as an upper bound. A direct way to operationalize the "recovery" term in the decomposition is to measure the frequency with which Route 2 succeeds while Route 3 fails on the *same* generated code trace, i.e.,

$$\Pr\left(\hat{Y}_{\text{Sim}} = Y^*(X),\ g(X, Z_{\text{C}}) \neq Y^*(X)\right).$$

This quantity corresponds to the *recovery mass* $\Pr(\neg C)\, r$ appearing in the above risk gap. It aggregates both (i) genuine recovery from incorrect code and (ii) cases where execution fails (e.g., $g(X, Z_{\text{C}}) = \perp$) but the simulator still answers correctly. Because the simulator may partially ignore Z_{C} and answer directly from X, this statistic should be interpreted as an *upper bound* on mechanistic "recovery from flawed code."

Interpretation. Route 2 can outperform Route 3 only if the recovery mass $\Pr(\neg C)\, r$ is large enough to offset simulation noise on correct-execution instances $\Pr(C)\, e_C$. Empirically, we find that recovery mass is consistently small across tasks and models (Figure 9), indicating that Route 2 rarely compensates for execution failures via recovery. Combined with the fact that e_C is very high in practice, this explains why deterministic execution typically achieves lower end-to-end error than simulation on the same generated code traces (Figure 3).

6. Related Work and Discussion

Neuro-symbolic Learning. This paper builds on research in neuro-symbolic integration (Graves et al., 2014; Veličković & Blundell, 2021; Reed & Freitas, 2016; Graves et al., 2016), which combines neural networks with symbolic reasoning systems. These approaches are motivated by cognitive science (Schneider & Chein, 2003; Risko & Gilbert, 2016; Anderson, 2010), hierarchical reinforcement learning (Kolter et al., 2007; Dietterich, 2000), and compositionality research (Hudson & Manning, 2018; Hupkes et al., 2020; Andreas et al., 2017; Poggio et al., 2017). An orthogonal line of work explores direct execution of algorithms by neural networks (Veličković & Blundell, 2021; Mahdavi et al., 2023; Ibarz et al., 2022; Yan et al., 2020). Unlike these approaches that focus on *how* to integrate neural and symbolic components, our work addresses *why* neuro-symbolic integration outperforms neural reasoning alone for algorithmic tasks

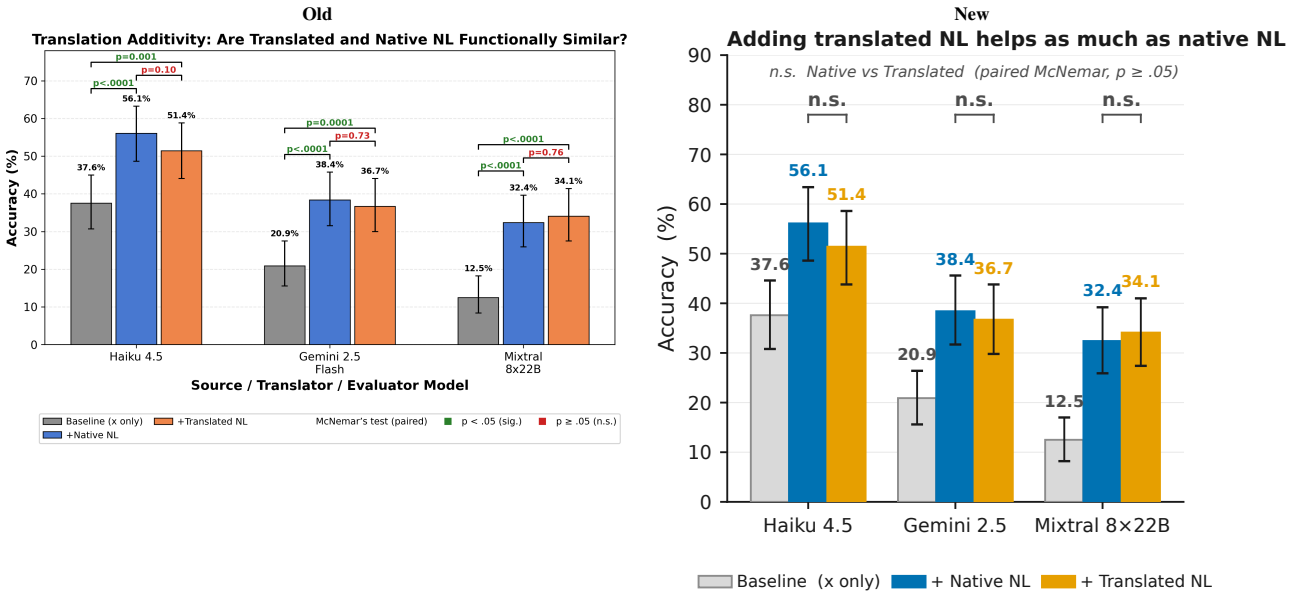

Figure 8. **Translated NL has the same functionality as the original native NL quantified by end-task accuracy.** When concatenating the reasoning traces to the prompt, we get much higher accuracy as opposed to the baseline with just the prompt, indicating the model is using the reasoning. There are statistically significant differences between the baseline and concatenated prompts, but no statistically significant difference between the Translated NL and Native NL concatenations. Source data is randomly sampled from existing data used to generate Figure 3, we use 1000 samples per model.

(Sections 4 and 5).

LLM Reasoning. Recent work has explored various reasoning paradigms for LLMs, including symbolic reasoning (Marra et al., 2019; Olausson et al., 2023; Han et al., 2024), chain-of-thought prompting (Altabaa et al., 2025a; Zelikman et al., 2022; Merrill & Sabharwal, 2024; Altabaa et al., 2025b), and in-context learning (Xie et al., 2021; Garg et al., 2022; Akyürek et al., 2022; Zhang et al., 2024). Xie et al. (2021) model in-context learning as implicit Bayesian inference, which we extend to compare different reasoning representations. While prior work demonstrates *that* certain prompting strategies improve performance, we provide a theoretical framework (Section 2) explaining *why* code representations lead to lower Bayes error (Proposition 4.2).

LLM Tool-Use. Tool-augmented LLMs have achieved strong empirical results (Shen, 2024; Schick et al., 2023; Qin et al., 2023; Tang et al., 2023; Parisi et al., 2022). Code generation for tool-use can be viewed as a form of semantic parsing (Shin & Durme, 2022; Krishnamurthy et al., 2017; Berant et al., 2013; Dong & Lapata, 2016) or function calling (Puri et al., 2021; Alon et al., 2019; Chen & Zhou, 2018). Our work complements this literature by providing theoretical justification (Proposition 4.2 and Section 5) for the observed empirical advantages of code-based tool-use over direct natural language reasoning.

7. Conclusion

We introduced a three-route framework (Section 2) for comparing code and natural language representations in algorithmic reasoning. By introducing an intermediate route—code generation with LLM-based simulation—we isolate translation effects from execution effects, enabling tractable pairwise analysis. Empirically, code execution achieves +28.9% v.s. NL across 48 algorithmic tasks and 6 models (Section 3.2). In our analysis (Section 4), we first show that under the optimal risk regime, code is at least as good as NL up to a small ε bound. Thus, when designing near-optimal end-to-end reasoning pipelines, trace generation with code will never be a bottleneck w.r.t. to natural language. We further verify in the second part of our analysis that execution is the bottleneck by showing that when the code is correct, execution always beats language reasoning ($>$ 60%), and when code is correct, language reasoning seldom beats execution ($<$2%), as shown in Figure 9. Overall, when designing stronger AI systems that solve harder tasks, understanding that language reasoning struggles to scale with hardness should motivate us to design compositional AI systems that plan and reason in code and delegate execution to external solvers, as opposed to monolithic architectures.

Limitations. (1) *Empirical approximations*: Our distributional similarity tests use finite samples and specific judge models, introducing estimation variance. Furthermore, there is room for optimization with the prompts of both judges and translators. (2) *Model coverage*: While we evaluate both frontier (GPT-4o, Claude, Gemini) and open-source models (Mistral, Codestral), we cannot guarantee findings generalize to all architectures or future models. (3) *Theoretical assumptions*: The statistical analysis result assumes the existence of a garbling channel from code to NL, which we validate empirically but do not prove from first principles.

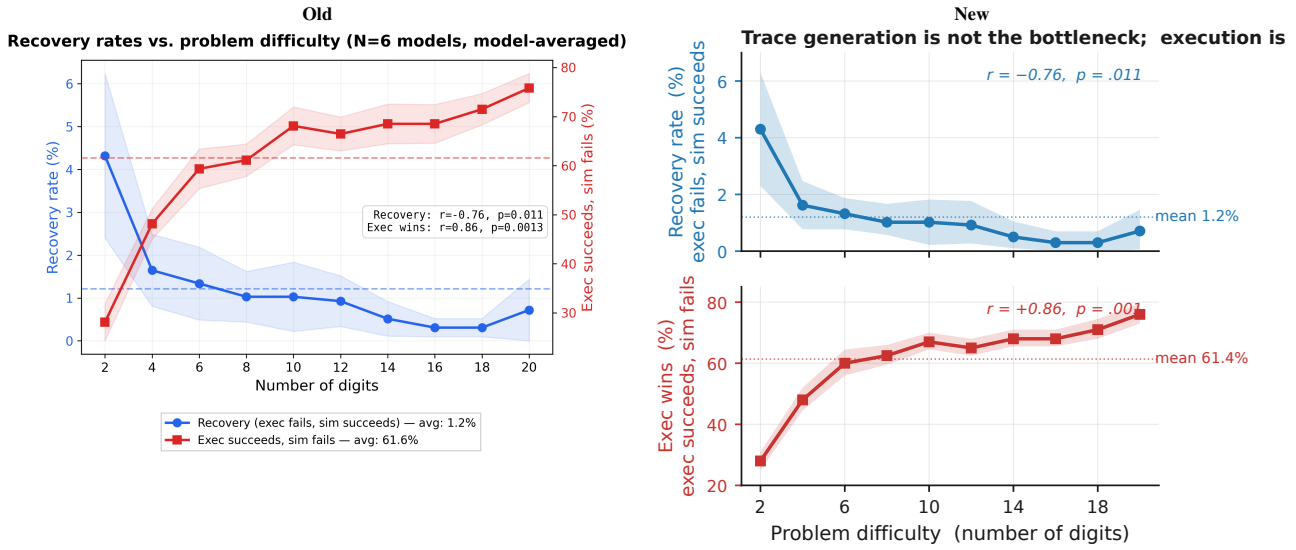

Figure 9. **Recovery mass (blue) $< 2\%$ on average and reduces as tasks get harder. On the other hand, LLM execution gets noisier compared to python execution (red)** . Recovery rate $\Pr(\neg C)\, r$ (Code Execution Fails but LLM Reasoning succeeds | Incorrect Code) in blue trends down. Exec wins $\Pr(C)e_C$ (Code Execution succeeds and LLM reasoning fails | Correct Code) increases as tasks get harder.

8. Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. What learning algorithm is in-context learning? investigations with linear models. *arXiv preprint arXiv:2211.15661*, 2022.

Alon, U., Zilberstein, M., Levy, O., and Yahav, E. code2vec: learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL): 1–29, January 2019. ISSN 2475-1421. doi: 10.1145/3290353. URL https://dl.acm.org/doi/10.1145/3290353.

Altabaa, A., Montasser, O., and Lafferty, J. Cot information: Improved sample complexity under chain-of-thought supervision. *arXiv preprint arXiv:2505.15927*, 2025a.

Altabaa, A., Montasser, O., and Lafferty, J. CoT Information: Improved Sample Complexity under Chain-of-Thought Supervision, May 2025b. URL http://arxiv.org/abs/2505.15927. arXiv:2505.15927 [stat].

Anderson, M. L. Neural reuse: A fundamental organizational principle of the brain. *Behavioral and Brain Sciences*, 33(4):245–266, August 2010. ISSN 0140-525X, 1469-1825. doi: 10.1017/S0140525X10000853. URL https://www.cambridge.org/core/product/identifier/S0140525X10000853/type/journal_article.

Andreas, J., Rohrbach, M., Darrell, T., and Klein, D. Neural Module Networks, July 2017. URL http://arxiv.org/abs/1511.02799. arXiv:1511.02799 [cs].

Anthropic. The claude 3 model family: A new standard for intelligence. *Anthropic Technical Report*, 2024.

Berant, J., Chou, A., Frostig, R., and Liang, P. Semantic Parsing on Freebase from Question-Answer Pairs. In Yarowsky, D., Baldwin, T., Korhonen, A., Livescu, K., and Bethard, S. (eds.), *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1533–1544, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL https://aclanthology.org/D13-1160/.

Chen, Q. and Zhou, M. A neural framework for retrieval and summarization of source code. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pp. 826–831, Montpellier France, September 2018. ACM. ISBN 978-1-4503-5937-5. doi: 10.1145/3238147.3240471. URL https://dl.acm.org/doi/10.1145/3238147.3240471.

Dietterich, T. G. Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, November 2000. ISSN 1076-9757. doi: 10.1613/jair.639. URL https://www.jair.org/index.php/jair/article/view/10266.

Dong, L. and Lapata, M. Language to Logical Form with Neural Attention, June 2016. URL http://arxiv.org/abs/1601.01280. arXiv:1601.01280 [cs].

Fan, L., Hua, W., Li, L., Ling, H., Zhang, Y., and Hemphill, L. Nphardeval: Dynamic benchmark on reasoning ability of large language models via complexity classes. *arXiv preprint arXiv:2312.14890*, 2023.

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. Pal: Program-aided language models. In *International Conference on Machine Learning*, pp. 10764–10799. PMLR, 2023.

Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. What can transformers learn in-context? a case study of simple function classes. *Advances in neural information processing systems*, 35:30583–30598, 2022.

Graves, A., Wayne, G., and Danihelka, I. Neural Turing Machines, December 2014. URL http://arxiv.org/abs/1410.5401. arXiv:1410.5401 [cs].

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., Colmenarejo, S. G., Grefenstette, E., Ramalho, T., Agapiou, J., Badia, A. P., Hermann, K. M., Zwols, Y., Ostrovski, G., Cain, A., King, H., Summerfield, C., Blunsom, P., Kavukcuoglu, K., and Hassabis, D. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, October 2016. ISSN 0028-0836, 1476-4687. doi: 10.1038/nature20101. URL https://www.nature.com/articles/nature20101.

Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Zhou, W., Coady, J., Peng, D., Qiao, Y., Benson, L., Sun, L., Wardle-Solano, A., Szabo, H., Zubova, E., Burtell, M., Fan, J., Liu, Y., Wong, B., Sailor, M., Ni, A., Nan, L., Kasai, J., Yu, T., Zhang, R., Fabbri, A. R., Kryscinski, W., Yavuz, S., Liu, Y., Lin, X. V., Joty, S., Zhou, Y., Xiong, C., Ying, R., Cohan, A., and Radev, D. FOLIO: Natural Language Reasoning with First-Order Logic, October 2024. URL http://arxiv.org/abs/2209.00840. arXiv:2209.00840 [cs].

Hudson, D. A. and Manning, C. D. Compositional Attention Networks for Machine Reasoning, April 2018. URL http://arxiv.org/abs/1803.03067. arXiv:1803.03067 [cs].

Hupkes, D., Dankers, V., Mul, M., and Bruni, E. Compositionality decomposed: how do neural networks generalise?, February 2020. URL http://arxiv.org/abs/1908.08351. arXiv:1908.08351 [cs].

Ibarz, B., Kurin, V., Papamakarios, G., Nikiforou, K., Bennani, M., Csordás, R., Dudzik, A. J., Bošnjak, M., Vitvitskyi, A., Rubanova, Y., Deac, A., Bevilacqua, B., Ganin, Y., Blundell, C., and Veličković, P. A Generalist Neural Algorithmic Learner. In *Proceedings of the First Learning on Graphs Conference*, pp. 2:1–2:23. PMLR, December 2022. URL https://proceedings.mlr.press/v198/ibarz22a.html. ISSN: 2640-3498.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Kolter, J., Abbeel, P., and Ng, A. Hierarchical Apprenticeship Learning with Application to Quadruped Locomotion. In *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc., 2007. URL https://proceedings.neurips.cc/paper_files/paper/2007/hash/54a367d629152b720749e187b3eaa11b-Abstract.html.

Krishnamurthy, J., Dasigi, P., and Gardner, M. Neural Semantic Parsing with Type Constraints for Semi-Structured Tables. In Palmer, M., Hwa, R., and Riedel, S. (eds.), *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 1516–1526, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1160. URL https://aclanthology.org/D17-1160/.

Lyu, Q., Havaldar, S., Stein, A., Zhang, L., Rao, D., Wong, E., Apidianaki, M., and Callison-Burch, C. Faithful chain-of-thought reasoning. In *The 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL 2023)*, 2023.

Mahdavi, S., Swersky, K., Kipf, T., Hashemi, M., Thrampoulidis, C., and Liao, R. Towards Better Out-of-Distribution Generalization of Neural Algorithmic Reasoning Tasks, March 2023. URL http://arxiv.org/abs/2211.00692. arXiv:2211.00692 [cs].

Marra, G., Giannini, F., Diligenti, M., and Gori, M. Integrating Learning and Reasoning with Deep Logic Models, January 2019. URL http://arxiv.org/abs/1901.04195. arXiv:1901.04195 [cs].

Merrill, W. and Sabharwal, A. The Expressive Power of Transformers with Chain of Thought, April 2024. URL http://arxiv.org/abs/2310.07923. arXiv:2310.07923 [cs].

Olausson, T. X., Gu, A., Lipkin, B., Zhang, C. E., Solar-Lezama, A., Tenenbaum, J. B., and Levy, R. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order

Logic Provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 5153–5176, 2023. doi: 10.18653/v1/2023.emnlp-main.313. URL http://arxiv.org/abs/2310.15164. arXiv:2310.15164 [cs].

Pan, L., Albalak, A., Wang, X., and Wang, W. Y. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. *arXiv preprint arXiv:2305.12295*, 2023.

Parisi, A., Zhao, Y., and Fiedel, N. TALM: Tool Augmented Language Models, May 2022. URL http://arxiv.org/abs/2205.12255. arXiv:2205.12255 [cs].

Poggio, T., Mhaskar, H., Rosasco, L., Miranda, B., and Liao, Q. Why and when can deep-but not shallow-networks avoid the curse of dimensionality: a review. *International Journal of Automation and Computing*, 14(5):503–519, 2017.

Puri, R., Kung, D. S., Janssen, G., Zhang, W., Domeniconi, G., Zolotov, V., Dolby, J., Chen, J., Choudhury, M., Decker, L., Thost, V., Buratti, L., Pujar, S., Ramji, S., Finkler, U., Malaika, S., and Reiss, F. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks, August 2021. URL http://arxiv.org/abs/2105.12655. arXiv:2105.12655 [cs].

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., and Sun, M. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs, October 2023. URL http://arxiv.org/abs/2307.16789. arXiv:2307.16789 [cs].

Reed, S. and Freitas, N. d. Neural Programmer-Interpreters, February 2016. URL http://arxiv.org/abs/1511.06279. arXiv:1511.06279 [cs].

Risko, E. F. and Gilbert, S. J. Cognitive Offloading. *Trends in Cognitive Sciences*, 20(9):676–688, September 2016. ISSN 13646613. doi: 10.1016/j.tics.2016.07.002. URL https://linkinghub.elsevier.com/retrieve/pii/S1364661316300985.

Schick, T., Dwivedi-Yu, J., Dessí, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: language models can teach themselves to use tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

Schneider, W. and Chein, J. M. Controlled & automatic processing: behavior, theory, and biological mechanisms. *Cognitive Science*, 27(3):525–559, May 2003. ISSN 0364-0213, 1551-6709. doi: 10.1207/s15516709cog2703_8. URL https://onlinelibrary.wiley.com/doi/10.1207/s15516709cog2703_8.

Shen, Z. LLM With Tools: A Survey, September 2024. URL http://arxiv.org/abs/2409.18807. arXiv:2409.18807 [cs].

Shin, R. and Durme, B. V. Few-Shot Semantic Parsing with Language Models Trained On Code, May 2022. URL http://arxiv.org/abs/2112.08696. arXiv:2112.08696 [cs].

Tang, Q., Deng, Z., Lin, H., Han, X., Liang, Q., Cao, B., and Sun, L. ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases, September 2023. URL http://arxiv.org/abs/2306.05301. arXiv:2306.05301 [cs].

Team, G., Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., Sorber, R., Schalkwyk, J., Dai, A. M., Hauth, A., et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Veličković, P., Badia, A. P., Budden, D., Pascanu, R., Banino, A., Dashevskiy, M., Hadsell, R., and Blundell, C. The clrs algorithmic reasoning benchmark. *International Conference on Machine Learning*, pp. 22084–22102, 2022.

Veličković, P. and Blundell, C. Neural algorithmic reasoning. *Patterns*, 2(7):100273, July 2021. ISSN 26663899. doi: 10.1016/j.patter.2021.100273. URL https://linkinghub.elsevier.com/retrieve/pii/S2666389921000994.

Wang, Z., Chang, Q., Patel, H., Biju, S., Wu, C.-E., Liu, Q., Ding, A., Rezazadeh, A., Shah, A., Bao, Y., and Siow, E. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers, 2025. URL https://arxiv.org/abs/2508.20453.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*, 2021.

Yan, Y., Swersky, K., Koutra, D., Ranganathan, P., and Hashemi, M. Neural Execution Engines: Learning to Execute Subroutines, October 2020. URL http://arxiv.org/abs/2006.08084. arXiv:2006.08084 [cs].

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=mXpq6ut8J3.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. STaR: Bootstrapping Reasoning With Reasoning, May 2022. URL http://arxiv.org/abs/2203.14465. arXiv:2203.14465 [cs].

Zhang, R., Frei, S., and Bartlett, P. L. Trained transformers learn linear models in-context. *Journal of Machine Learning Research*, 25(49):1–55, 2024.

A. Supplementary Route Results

In all route tables, Route 1 is natural-language reasoning, Route 2 is natural-language simulation over a code trace, and Route 3 is deterministic code execution. Captions and result paragraphs state whether a table uses the full evaluation set, a 25% subset, a single seed, or per-arm parse-normalized denominators.

A.1. Complexity-Stratified Route Accuracy

We stratify the main route-accuracy evaluation by the asymptotic complexity class of each task. Across classes, Route 3 remains the main source of improvement over Route 2, while Route 2 is close to Route 1.

Complexity	Tasks	Inst.	R1	R2	R3	R2–R1	$p_{2,1}$	R3–R2
$O(1)$	4	227	49.8	49.4	85.0	-0.4	0.52	35.6
$O(\log n)$	1	35	22.4	28.6	32.9	6.2	2.7e-04	4.3
$O(n)$	4	135	4.8	5.0	7.5	0.2	0.635	2.5
$O(n \log n)$	5	68	0.0	0.0	0.0	0.0	1	0.0
$O(n^2)$	17	357	12.4	12.0	49.0	-0.4	0.157	37.0
$O(n^2 \log n)$	1	1	0.0	0.0	0.0	0.0	1	0.0
$O(n^3)$	4	37	0.0	0.0	0.0	0.0	1	0.0
NP-hard	8	480	23.8	21.7	59.2	-2.1	5.0e-09	37.5

Table 1. Route accuracy grouped by asymptotic complexity.

Result. Accuracies and deltas in Table 1 are percentages or percentage points. McNemar tests are exact two-sided tests on paired outcomes. The main route pattern is not an artifact of averaging together easy and hard task families: the Route 3–Route 2 gain remains the largest observed change in the high-support classes, while Route 2 stays close to Route 1.

Task mapping. The largest strata are $O(n^2)$ dynamic-programming, sorting, string, graph, and shortest-path tasks; and NP-hard ILP, graph coloring, edge-disjoint paths, shortest path, and TSP tasks. The smaller strata cover constant-time arithmetic and geometry, logarithmic binary search, linear selection/string matching, $O(n \log n)$ scheduling/sorting/hulls, $O(n^2 \log n)$ Kruskal MST, and cubic dynamic-programming/shortest-path routines.

A.2. Model-Stratified Route Accuracy

We stratify the main route comparison by model to check whether the aggregate result is driven by one model family or scale regime. The Route 3 advantage persists across all six full-coverage models, while the Route 2–Route 1 difference is small and changes sign across models.

Model	Type	Inst.	R1	R2	R3	R2–R1	$p_{2,1}$	R3–R2	$p_{3,2}$
anthropic/claude-haiku-4.5	closed	1340	31.4	27.7	48.2	-3.7	4.4e-13	20.5	6.6e-123
google/gemini-2.0-flash-001	closed	1340	22.3	21.1	54.4	-1.2	0.0159	33.3	$< 10^{-300}$
google/gemini-2.5-flash	closed	1340	23.4	23.5	54.3	0.0	0.961	30.8	1.5e-277
openai/gpt-4o-mini	closed	1340	18.2	16.8	49.6	-1.4	0.00312	32.8	2.7e-280
mistralai/codestral-2508	open	1340	17.6	18.9	52.3	1.2	0.00113	33.5	1.3e-284
mistralai/mixtral-8x22b-instruct	open	1340	15.2	15.7	42.8	0.5	0.143	27.1	3.0e-221

Table 2. Route accuracy grouped by model.

Result. Each row in Table 2 has 1,340 unique problems and 4,020 paired route evaluations. Accuracies and deltas are percentages or percentage points. This stratification shows that the execution gain is not an artifact of mixing one weak or strong model into the aggregate.

B. Functional Similarity Checks

B.1. Prompt Translation Shot Ablation

The functional similarity test depends on translating code traces back into natural language. To test sensitivity to prompting, we vary the number of in-context examples used by the translator. More examples improve translated-trace utility and narrow the gap to native natural-language traces.

Shots	x	x + native NL	x + translated NL	Δ native	Δ translated	Gap
0	32.70%	55.70%	42.41%	+23.00pp	+9.70pp	-13.29pp
1	32.70%	55.70%	45.78%	+23.00pp	+13.08pp	-9.92pp
2	32.70%	55.70%	49.58%	+23.00pp	+16.88pp	-6.12pp
3	32.70%	55.70%	48.52%	+23.00pp	+15.82pp	-7.17pp
4	32.70%	55.70%	49.37%	+23.00pp	+16.67pp	-6.33pp
5	32.70%	55.70%	50.00%	+23.00pp	+17.30pp	-5.70pp
10 (reference)	39.00%	56.50%	52.00%	+17.50pp	+13.00pp	-4.50pp

Table 3. Translation-additivity shot ablation for Claude Haiku 4.5.

Result. Rows 0–5 in Table 3 use a matched 25% subset sweep. The 10-shot reference row comes from the original main-evaluation setting and is not from the same subset sweep. Here, x is the question alone, x + native NL appends the original natural-language trace, and x + translated NL appends the natural-language trace translated from code.

B.2. Recursive Language Model Evaluation

To broaden coverage beyond standard LLM forward passes, we evaluate recursive language model execution on a 25% subset at seed 0. Route 1 and Route 2 remain close under this executor family.

Model	RLM Code Acc	RLM NL Acc	Better Arm
Claude Haiku 4.5	29.4%	30.9%	NL
Gemini 2.5 Pro	47.5%	45.2%	Code

Table 4. Recursive language model results on the 25% subset, seed 0.

Result. The corresponding counts are 202/686 code-correct vs. 212/686 NL-correct for Claude Haiku 4.5, and 326/686 code-correct vs. 310/686 NL-correct for Gemini 2.5 Pro. Because this is a 25% subset, we use it as model-coverage evidence rather than as the main route estimate.

C. Additional Model Evaluations

C.1. Coding-Specialized Model Evaluation

We evaluate coding-specialized models to test whether models trained on code show a different Route 2 pattern. Even for these models, replacing natural-language traces with code traces changes little compared with replacing LLM simulation by actual code execution.

Model	NL	Sim	Code Exec
x-ai/grok-code-fast-1 (25% data)	47.71% (167/350)	47.71% (167/350)	55.99% (159/284)
qwen/qwen3-coder (25% data)	30.23% (104/344)	26.86% (94/350)	56.19% (168/299)
codestral-2508 (original)	19.89% (943/4740)	23.14% (1097/4740)	59.65% (2266/3799)

Table 5. Route accuracy for coding-specialized models. Each cell reports accuracy over correct / denominator.

Result. Grok and Qwen use 25% subset runs; Codestral uses the original evaluation run. Rows use per-arm parse-normalized denominators: NL and Sim drop rows where that arm failed to parse, while Code Exec keeps rows with successful code execution. The 25% subset rows broaden model coverage but are not pooled into the main six-model estimate.

C.2. Frontier Model Controlled-Subset Evaluation

We also evaluate two frontier models on a 350-instance subset at seed 1 with tool calling disabled and no patching or repair step. These results are not used as the main estimate because they are smaller than the full six-model evaluation, but they provide a controlled check that the route pattern is not explained only by explicit tool calls.

Model	Route 1	Route 2	Route 3
GPT-5.4	42.57% (149/350)	41.43% (145/350)	54.01% (175/324)
Claude Opus 4.6	52.73% (174/330)	73.42% (116/158)	77.54% (107/138)

Table 6. Frontier controlled-subset route accuracy. Entries are accuracy (correct / denominator).

Result. Reasoning-mode runs are excluded because hidden reasoning and tool scaffolding confound the comparison; rows use seed 1, tool calling disabled, and per-arm parse-normalized denominators.

D. Failure Analysis

D.1. Code Execution Failure Modes

When Route 3 fails, failures are mostly semantic wrong answers rather than syntax or timeout failures. This supports the interpretation that code execution errors often originate in the generated program specification rather than in the Python runtime itself.

Model	Wrong answer	Syntax error	Runtime error	Time limit
Haiku	50.60%	16.37%	2.80%	30.23%
Codestral	63.58%	4.36%	28.62%	3.44%
Gemini 2.0	71.79%	19.34%	2.46%	6.41%
Gemini 2.5	73.36%	15.62%	3.38%	7.64%
GPT-4o mini	65.82%	3.60%	26.73%	3.85%
Mixtral	45.77%	5.56%	46.04%	2.63%
Total	61.06%	10.54%	19.23%	9.17%

Table 7. Distribution of Route 3 code-execution failures.

Result. Table 7 is conditional on Route 3 failures and excludes parse-only rows where execution completed but the returned value failed parsing or type checks. This keeps the diagnostic focused on what happened after code execution was actually attempted.

E. Reproducibility Notes

The appendix tables are generated by the accompanying analysis code.

Route tables. `analyze_route_accuracy_tables.py` uses the full six-model route evaluation and paired route outcomes for McNemar tests.

Translation shot ablation. `analyze_translation_shot_ablation.py` generates the matched 25% subset sweep for rows 0–5; the 10-shot row is retained as the original main-evaluation reference.

Additional evaluators. `analyze_rlm_subset_results.py` covers the 25% subset recursive-language model runs, `analyze_coding_model_table.py` covers the coding-specialized model rows, and `analyze_frontier_nopatch_table.py` covers the 350-instance frontier controlled subset.

Failure modes. `analyze_code_failure_distribution.py` reports Route 3 failures after excluding parse-only rows.